实验六：Shell 自动化编程与 ECC 编译工具链

- **适用课程**：Linux 操作系统
- **推荐学时**：3 学时（讲授 1 学时，练习与实战 2 学时）
- **实验平台**：Earendel Web OS [https://earendel.techarts.cn]

实验目标

1. **掌握 Shell 核心编程技能**：熟练掌握 Shell 变量、参数引用、控制结构（ `if` / `for` / `while` ）、函数与数组，能够编写结构清晰的自动化脚本。
2. **掌握退出状态码与数据管道流**：深入理解 `$?` 状态码在流程控制中的核心作用，熟练结合管道与文本工具（ `grep` 、 `cut` 、 `sort` 、 `wc` 、 `tee` ）处理输出数据。
3. **理解可执行文件结构与编译工具链**：掌握 ECC 编译器的核心参数（ `-o` 、 `-O2` 、 `-s` ），理解 EAF 文件组织结构及其与 Linux ELF 的设计思想映射。
4. **构建自动化发布流水线（CI/CD）**：编写工程化 Shell 构建脚本，实现源码校验、编译打包、状态拦截、测试验收与报告生成的自动化闭环。

Part A：核心知识点解析与动手实验

1. Shell 编程核心语法精讲

在终端中创建并进入本实验专属目录：

```
mkdir -p ~/lab6 && cd ~/lab6
```

(1) 变量、引用与输出重定向

- **变量定义与引用**：变量名与等号之间**不能有空格**。引用变量使用 `$VAR` 或 `${VAR}` 。
- **单引号与双引号的区别**：
  - **双引号 `""`** ：允许变量解析和命令替换（弱引用）。
  - **单引号 `''`** ：保留所有字符的字面值，不解析变量（强引用）。
- **重定向**： `>` 覆盖写入， `>>` 追加写入。

动手练习（创建 `01_basics.sh` ）：

```bash
#!/bin/bash
APP="Earendel_App"
VER="1.0.0"

# 1. 观察引号差异
echo "双引号解析：项目 $APP 当前版本 $VER"
echo '单引号原样输出：项目 $APP 当前版本 $VER'

# 2. 输出重定向测试
echo "=== 构建日志开始 ===" > lab.log
echo "记录一条状态数据" >> lab.log
cat lab.log
```

> 运行方式： `bash 01_basics.sh` 或赋予权限后执行： `chmod +x 01_basics.sh && ./01_basics.sh` 。

(2) 条件分支与命令退出状态码 (`$?`)

在 Linux 运维脚本中，任何一条命令执行完毕都会返回一个退出状态码（Exit Code）：

- `0`：表示命令**执行成功**。
- **非 0**（1–255）：表示命令**执行失败或出现异常**。
- 环境变量 `$?` 永远保存**上一条命令**的退出状态码。

动手练习（创建 `02_condition.sh` ）：

```bash
#!/bin/bash
SCORE=85

# 1. 条件判断与目录检查 (-d 判断目录是否存在)
if [ ! -d "results" ]; then
  mkdir -p results
  echo "创建了 results 目录"
fi

# 2. 数值对比判断 (-ge: 大于等于, -lt: 小于)
if [ $SCORE -ge 60 ]; then
  echo "成绩合格: $SCORE 分"
else
  echo "成绩不合格: $SCORE 分"
fi

# 3. 状态码捕获测试
ls lab.log > /dev/null
echo "上一条成功命令的状态码: $?"   # 输出 0

ls not_exist_file.txt > /dev/null 2>&1
echo "上一条失败命令的状态码: $?"   # 输出非 0
```

> **Important**
>
> **关键原则**：如果要对命令的退出状态做逻辑判断，必须在命令执行后**紧接着读取 `$?`**。如果在中间插入了其他命令（例如 `echo`），`$?` 会被刷新为后续命令的状态。

(3) 循环结构与数组

- `for` 循环：用于遍历列表或集合。
- `while` 循环：用于基于条件的循环控制。
- 数组操作：`${arr[@]}` 获取全部元素，`${#arr[@]}` 获取数组长度。

动手练习（创建 `03_loops.sh`）：

```bash
#!/bin/bash
# 1. for 循环遍历发布阶段
PHASES=("Lint" "Compile" "Test" "Package")

echo "执行流水线阶段（数组长度: ${#PHASES[@]}）: "
for stage in "${PHASES[@]}"
do
  echo "-> 正在执行: $stage"
done

# 2. while 循环累加计算
COUNT=1
SUM=0
while [ $COUNT -le 10 ]
do
  SUM=$((SUM + COUNT))
  COUNT=$((COUNT + 1))
done
echo "1 到 10 累加总和为: $SUM"
```

(4) Shell 函数与模块化封装

Shell 函数通过 `$1` 、 `$2` 等位置参数接收调用参数，函数中的 `return` 用于返回状态码（0–255）。

动手练习（创建 `04_functions.sh` ）：

```bash
#!/bin/bash
# 定义打印状态的函数
log_status() {
  local module=$1
  local status=$2
  echo "[$(date '+%Y-%m-%d %H:%M:%S')] 模块 [$module] 状态: $status"
}

# 调用函数
log_status "Kernel" "Running"
log_status "Storage" "OK"
```

(5) 文本流处理与管道综合练习

运维自动化中常使用文本工具处理编译日志与系统记录：

- `grep`： 过滤匹配文本。
- `cut`： 按列提取字段。
- `sort` / `uniq`： 排序与去重。
- `wc -l`： 统计行数。
- `tee`： 既将数据输出到终端，又保存到文件。

动手练习：
准备测试日志 `builds.txt`：

```
echo "2026-10-01 compile success" > builds.txt
echo "2026-10-01 test success" >> builds.txt
echo "2026-10-02 compile failed" >> builds.txt
echo "2026-10-03 compile success" >> builds.txt
echo "2026-10-03 test failed" >> builds.txt
```

执行以下文本分析流水线：

```
# 1. 筛选出所有失败记录并统计次数
grep "failed" builds.txt | wc -l

# 2. 提取并去重所有出现过的日期列表
cut -d' ' -f1 builds.txt | sort | uniq

# 3. 提取失败日志，屏幕展示的同时保存到 failed-builds.txt
grep "failed" builds.txt | tee failed-builds.txt
```

2. ECC 编译器与 EAF 可执行文件原理

(1) ECC 编译器介绍

在真实 Linux 环境中，开发者使用 GCC/Clang 编译 C/C++ 生成二进制 ELF 文件；而在 Earendel Web OS 中，系统内置了教学编译工具链 **ECC**（Earendel Compiler Collection）。

ECC 负责将 JavaScript 应用源码及静态模块打包链接为 Earendel 原生可执行文件格式 —— **EAF**（Earendel Application Format）。

ECC 的基本命令格式：

```
ecc <入口源文件.js> -o <输出文件.eaf> [编译选项]
```

常用选项参数：

- `-o <文件路径>`：指定编译输出文件名。如果不带 `-o`，默认输出 `a.out.eaf`。
- `-O2`（或 `-O`）：启用代码优化（在当前实现中，会自动剥离无用的源码注释，精简代码体积）。
- `-s` / `--strip`：剥离符号表（去除调试与导出符号信息，大幅缩减文件大小）。
- `--embed <文件/目录>`：将静态配置文件或资源嵌入到生成的可执行文件中。

(2) EAF 文件结构剖析

EAF 借鉴了经典 Unix ELF（Executable and Linkable Format）文件的分段设计思想，采用分节（Sections）组织：

```
┌──────────────────────────────── EAF 可执行结构 ────────────────────────────────┐
│ magic: "EAF01"                 格式幻数（识别标志）                            │
│ version: "1.0.0"               格式版本                                        │
├─ header ──────────────────────────────────────────────────────────────────────┤
│ entry: "/home/hello/app.js"    程序入口源文件虚拟路径                          │
│ type: "EXEC"                   文件类型（可执行文件）                          │
│ arch: "js-vm"                  目标运行架构（Earendel 虚拟机）                 │
│ createdAt                      生成时间戳                                      │
├─ sections ────────────────────────────────────────────────────────────────────┤
│ .text                           代码段：经编译静态链接后的 JS 核心代码         │
│ .data                           数据段：应用元数据、配置与嵌入资源             │
│ .imports                        导入段：声明调用的系统与环境接口列表           │
│ .symtab                         符号表：导出函数与符号信息（-s 选项可剥离）    │
└────────────────────────────────────────────────────────────────────────────┘
```

EAF 与传统 ELF/PE 的概念对照：

结构概念	Linux ELF 对应物	Earendel EAF 对应内容	教学功能说明
文件标识	ELF Magic Number (`0x7F 'E' 'L' 'F'`)	`magic: "EAF01"`	标识可执行文件格式类型

结构概念	Linux ELF 对应物	Earendel EAF 对应内容	教学功能说明
文件头部	ELF Header (入口地址、目标架构)	`header` 字段 (`entry` , `arch`)	记录运行入口与架构环境
代码段	`.text` Section (机器指令)	`.text` Section (执行代码)	存放实际由系统执行的核心逻辑
数据段	`.data` Section (全局数据/资源)	`.data` Section (含 embeds)	存放嵌入的静态数据与运行时参数
符号表	`.symtab` / `.strtab`	`.symtab` Section	调试和导出符号，可用 `-s` 剥离

3. ECC 编译对比实验

在 `~/lab6` 下创建测试代码目录 `ecc-demo`：

```
mkdir -p ~/lab6/ecc-demo && cd ~/lab6/ecc-demo
```

编写一段带有注释的测试程序 `hello.js`：

```
// 项目名称: Earendel ECC 教学程序
// 开发者: Linux Lab
const title = "Linux Shell & Compiler Lab";
const version = 1.0;

console.log("=== " + title + " ===");
console.log("Current Version: " + version);
```

实验 1：常规编译与运行（体验 `-o`）

```
mkdir -p build
ecc hello.js -o build/hello.eaf
ls -l build
./build/hello.eaf
```

> **现象观察**：编译成功后生成 `hello.eaf` 。直接通过 `./build/hello.eaf` 即可像运行 Linux 二进制程序一样执行它。

> *思考*：尝试直接运行 `ecc hello.js`，查看默认生成的程序名是什么？

实验 2：代码优化对比（体验 `-O2`）

```
# 默认未优化编译
ecc hello.js -o build/hello-debug.eaf

# 开启 -O2 优化编译
ecc hello.js -O2 -o build/hello-o2.eaf

# 检查两个 EAF 文件中是否保留了注释
grep "教学程序" build/hello-debug.eaf
grep "教学程序" build/hello-o2.eaf
```

> **现象分析**：`-O2` 优化去除了源文件注释，使代码体积更加紧凑，而程序输出与功能不受任何影响。

实验 3：符号表剥离（体验 `-s` 瘦身）

```
# 普通编译（保留符号表）
ecc hello.js -o build/with-sym.eaf

# 使用 -s 剥离符号表
ecc hello.js -s -o build/stripped.eaf

# 检查符号表是否存在
grep "symtab" build/with-sym.eaf
grep "symtab" build/stripped.eaf
```

> **现象分析**：加入 `-s` 编译后，生成的 EAF 文件不再包含 `.symtab` 分节，进一步降低了文件体积。

实验 4：编译错误捕获与返回值验证

```
# 1. 尝试编译一个不存在的源文件
ecc missing.js -o build/err.eaf
echo "错误文件编译返回值：$?"     # 观察输出非 0

# 2. 传递非法参数
ecc hello.js --target=unknown -o build/err.eaf
echo "非法参数编译返回值：$?"     # 观察输出非 0
```

> **核心收获**：ECC 在遇到语法错误、参数错误或文件缺失时会返回非 0 状态码，这为编写自动化构建脚本提供了精准的条件拦截依据。

Part B：企业级实战——自动化构建与发布流水线（CI/CD）

1. 业务背景

在现代软件工程中，任何代码发布都遵循**自动化流水线（CI/CD Pipeline）**。一个合格的运维发布脚本必须具备：

1. **前置环境与源码检查**：文件不存在时立即阻断并报警。
2. **构建过程状态拦截**：编译失败时严禁发布上线。
3. **冒烟测试与验收**：编译完成后自动执行并校验运行结果。
4. **生成审计报告**：记录构建时间、选项参数和最终状态。

2. 准备项目源代码

在 `~/lab6/apollo` 目录下创建待发布的主程序 `app.js`：

```
mkdir -p ~/lab6/apollo && cd ~/lab6/apollo
```

编写 `app.js`：

```
// 航天工程 Apollo 控制模块
const mission = "Apollo Moon Landing";
const launchYear = 2026;

console.log("Mission Name: " + mission);
console.log("Launch Year : " + launchYear);
console.log("Status      : All Systems Nominal");
```

3. 编写发布流水线脚本 `deploy.sh`

在 `~/lab6/apollo/deploy.sh` 中实现完整构建流程：

```
#!/bin/bash
# ================================================
# Apollo 工程自动化构建与发布流水线
# ================================================

SOURCE="app.js"
BUILD_DIR="build"
OUTPUT="build/apollo.eaf"
REPORT="build-report.txt"

echo "========================================="
echo "开始执行 Apollo 自动化构建流水线"
echo "========================================="

# 步骤 1：创建构建输出目录
mkdir -p $BUILD_DIR

# 步骤 2：源码合规检查（阻断式设计）
if [ ! -f "$SOURCE" ]; then
  echo "[ERROR] 源码文件 $SOURCE 不存在，流水线终止！"
  exit 1
fi
echo "[PASS] 源码检查通过：$SOURCE"

# 步骤 3：调用 ECC 编译工具链（开启 -O2 优化并剥离符号表 -s）
echo "[BUILD] 正在调用 ECC 编译打包..."
ecc $SOURCE -O2 -s -o $OUTPUT
BUILD_STATUS=$?

if [ $BUILD_STATUS -ne 0 ]; then
  echo "[ERROR] ECC 编译失败（状态码：$BUILD_STATUS），拒绝发布！"
  echo "Build Failed at $(date)" > $REPORT
  exit 2
fi
echo "[PASS] 编译成功，生成目标文件：$OUTPUT"

# 步骤 4：自动化冒烟测试（执行生成的目标文件）
echo "[TEST] 正在启动可执行程序进行功能自测..."
./$OUTPUT
TEST_STATUS=$?

if [ $TEST_STATUS -ne 0 ]; then
  echo "[ERROR] 程序运行自测失败（状态码：$TEST_STATUS）！"
```

```
  exit 3
fi
echo "[PASS] 冒烟测试全部通过！"

# 步骤 5：生成构建归档报告
echo "========================================" > $REPORT
echo "Apollo CI/CD 构建报告" >> $REPORT
echo "构建时间：$(date)" >> $REPORT
echo "入口源码：$SOURCE" >> $REPORT
echo "输出文件：$OUTPUT" >> $REPORT
echo "编译参数：-O2 -s" >> $REPORT
echo "流水线状态：SUCCESS" >> $REPORT
echo "========================================" >> $REPORT

echo "========================================"
echo "流水线执行完毕，构建报告已生成至：$REPORT"
echo "========================================"
```

4. 赋予权限并验证构建

```
# 赋予可执行权限
chmod +x deploy.sh

# 执行流水线构建
./deploy.sh

# 查看最终生成的构建报告
cat build-report.txt
```

5. 故障注入与健壮性测试

为了验证脚本的健壮性，请模拟以下运维突发事故：

1. **测试用例 1：源文件丢失测试**

    ```
    mv app.js app.js.bak
    ./deploy.sh
    echo "流水线退出码：$?"
    ```

预期现象：脚本检测到源码丢失，立即报错退出，退出码为 `1`，且不会执行后续编译。
恢复操作：`mv app.js.bak app.js`

2. **测试用例 2：语法/编译错误阻断测试**
   修改 `app.js`，在第一行写入一段非法代码（如 `import "unknown_lib";` 或直接写入乱码字符）。

```
./deploy.sh
echo "流水线退出码：$?"
```

预期现象：ECC 编译报错，`BUILD_STATUS` 捕获到非零状态码，脚本直接在步骤 3 终止，拒绝运行有缺陷的程序。
恢复操作：修好 `app.js`，重新运行 `./deploy.sh` 恢复绿色构建状态。

思考与延伸

1. **状态码时效性**：在 Shell 脚本中，为什么必须在目标命令执行后**立刻**将 `$?` 赋值给变量？如果中间执行了 `echo` 会发生什么？
2. **编译器参数权衡**：ECC 的 `-O2` 与 `-s` 选项分别改变了输出文件中的什么内容？在生产部署时为什么通常建议同时开启它们？
3. **架构类比**：EAF 文件的 `.text` 和 `.data` 段分别存放什么？它与 Linux 系统下的 ELF 格式在设计思想上有何异同？
4. **运维实践延伸**：如果需要让这个构建流水线在服务器上每天凌晨 2 点自动拉取代码并构建，你应该如何配置 `crontab` 定时任务？

实验报告与提交规范

1. 提交文件清单

- `01_basics.sh`、`02_condition.sh`、`03_loops.sh`、`04_functions.sh` 脚本文件。
- `builds.txt` 与提取后的 `failed-builds.txt`。
- ECC 参数对比实验记录（记录 `-O2` 与 `-s` 在 `grep` 下的实际效果差异截图或终端输出）。
- `deploy.sh` 源码文件及成功生成的 `build-report.txt`。
- 故障注入测试的终端执行截图与退出码记录。

2. 评分要点（总分 100 分）

- **Shell 基础语法与文本流（30 分）**：变量引用、条件分支、循环结构与管道文本处理完整无误。

- **编译工具链理解与实验（30 分）**：熟练运用 ECC 参数，准确说明 `-o` 、 `-O2` 、 `-s` 对 EAF 结构的实际影响。
- **CI/CD 自动化流水线实现（30 分）**： `deploy.sh` 逻辑严密，正确拦截各阶段错误并规范生成报告。
- **故障注入测试与实验结论（10 分）**：故障复现真实可靠，思考题回答准确。